

ArchSync: A Controlled Feasibility Study of Evidence-Backed Architecture Drift Detection in TypeScript Systems

Vo Duc Hieu*, Tran Minh Hoang[†], Ha Hoang Bach[‡], Le Van Kiet[§],
Hoang Nguyen The[¶], and Minh Tam Phan[¶]

*Software Engineering, FPT University, Ho Chi Minh City, Vietnam
Email: voduchieu42@gmail.com; ORCID: 0009-0007-5389-5177

[†]Computer Science and Engineering, VNUK Institute for Research and Executive Education,
The University of Danang, Da Nang, Vietnam
Email: andyjobs2023@gmail.com; ORCID: 0009-0000-0302-1841

[‡]Information Assurance, FPT University, Ho Chi Minh City, Vietnam
Email: hahoangbach2005@gmail.com; ORCID: 0009-0000-5118-0660

[§]Software Engineering, VNUK Institute for Research and Executive Education,
The University of Danang, Da Nang, Vietnam
Email: levankiet1212.2004@gmail.com; ORCID: 0009-0007-8434-882X

[¶]Faculty of Software Engineering, FPT University HCMC,
Ho Chi Minh City, 70000, Vietnam
Email: hoangnt20@fe.edu.vn, tamprm@fe.edu.vn

Abstract—Software teams can keep builds green while implementation relationships drift away from an approved architecture, leaving reviewers without a repeatable way to decide whether a change is harmless, forbidden, or a legitimate evolution. This paper presents ArchSync, a deterministic architecture-conformance prototype that treats a machine-readable model as an executable contract and reconstructs an observed service graph from TypeScript source code. Stack-specific analyzers detect selected HTTP, PostgreSQL, Redis, and AMQP interactions, preserve file-level evidence, and feed a Git-diff gate that maps introduced findings to PASS, BLOCK, or REVIEW without rewriting the approved model. We verify the prototype on two co-developed development datasets and replay the same controlled changes through the incremental gate. Within these finite regression sets, the implementation agrees with the declared graph, decision, evidence-location, and full-scan oracles, and provenance tracking removes the targeted detector errors exposed by the earlier analyzer. An external-tool comparison remains proposed and was not executed within the reported evaluation; no accepted comparator result or common label mapping is available, and the study lacks an independently curated real-world holdout. Its results therefore establish executable-contract coherence and reproducible regression behavior only, not comparative advantage or general accuracy on TypeScript systems.

Index Terms—software architecture, architecture conformance, architecture drift, static analysis, architecture as code

I. INTRODUCTION

Software architecture describes a system’s intended components, responsibilities, and permitted relationships. Implementation can diverge from this intent. Mapping work re-

ports structural violations and effects on software quality and evolution; developer evidence identifies continuing reliance on documentation, clear responsibilities, and shared practices [1], [2]. A local change can bypass a gateway, couple an interface directly to a database, or add infrastructure without an architecture decision while compilation and functional tests pass.

Without executable checks, documentation can leave a gap between intended and implemented structure. ArchSync connects architecture intent and implementation through deterministic, evidence-backed checks. Diagrams communicate intent but are difficult to use as automated gates; source-derived graphs expose relationships but do not specify which are allowed, forbidden, required, or subject to review.

We present ArchSync, an architecture-conformance workflow based on two graphs. The expected graph is derived from a versioned `architecture.yaml` document containing components, relationships, and deterministic rules. The observed graph is inferred from source code by stack-specific analyzers. ArchSync compares the graphs and produces one of three classifications: *no-impact* when implementation conforms without topology changes, *violation* when one or more rules are broken, and *evolution* when a non-forbidden topology change requires human review. Findings retain both the model location that establishes the expectation and the source location supporting the observation.

For pull requests, the workflow compares findings at the Git merge base with findings at the proposed head. Only findings introduced by the change determine its gate decision:

violations block, non-forbidden topology changes request review, and changes without new drift pass. The observed graph therefore follows implementation changes, while the expected graph remains the approved design baseline. Our proposed governance workflow requires an explicit model change and architecture-owner review to accept evolution; the evaluated command reports the need for review but does not manage approval or rejection records.

The prototype deliberately focuses on TypeScript and Node.js. It recognizes HTTP calls through `fetch`, PostgreSQL access through `pg`, Redis operations, and AMQP publishing and consumption. This is an empirical boundary rather than a claim that architecture drift is TypeScript-specific: the expected graph, rule engine, finding contract, and metrics are language-neutral, while each additional language requires a separately implemented and evaluated source-to-graph analyzer. The bounded TypeScript scope permits deterministic contracts and reproducible measurement without probabilistic inference. Large-language-model explanations and automatic repairs are excluded from the decision path.

This paper makes the following contributions:

- an executable-contract prototype integrating typed service relationships, four conformance rules, and source/model evidence;
- a relation-level Git-diff gate that separates introduced violations, reviewable evolution, and historical findings using cached component-scoped observation;
- a reproducible controlled verification artifact comprising 20 patches and 40 detector signals, with explicit units, provenance, and limits of inference.

These are bounded prototype contributions, not claims that expected-versus-observed comparison, dependency rules, or deterministic analysis are new concepts.

II. BACKGROUND AND RELATED WORK

This section is a scoped narrative synthesis selected to establish context for ArchSync’s design, not a systematic literature review. Sources were drawn from the retained keyword-search corpus and selected for their relevance to architecture conformance, drift detection, and static-analysis reliability, with emphasis on 2022–2025 publications. Four earlier works are retained as concept-origin references. Selection was purposive rather than exhaustive; the discussion distinguishes evaluated techniques, tool demonstrations, and secondary syntheses, and does not establish the absence of competing approaches.

A. Architecture Erosion and Reconstruction

Architecture erosion denotes progressive misalignment between intended architecture and implementation. A mapping study of 73 primary studies characterizes its manifestations and consequences [1]. Evidence from two OpenStack projects identifies erosion symptoms in code reviews [3]; a subsequent journal study examines violation symptoms and developer responses across OpenStack and Qt projects [4]. Their overlapping contexts are not independent replications. A mixed-methods study further identifies documentation, clear respon-

sibilities, and shared practices as ways developers manage drift [2]. They motivate implementation-linked checks, not a single metric of architecture quality.

Reconstruction recovers architectural views from implementation artifacts. Ducasse and Pollet’s foundational taxonomy distinguishes bottom-up, top-down, and hybrid processes [5]. A recent tertiary mapping organizes the applicability of recovery approaches, emphasizing that selecting an appropriate method depends on its context and assumptions [6]. A microservice-focused mapping compares the purposes, inputs, and outputs of reconstruction tools and identifies availability and maintenance concerns [7]. View-based drift analysis demonstrates how reconstructed specifications can be compared with intended models in an industrial case study [8]. ArchSync adopts a narrower observation boundary: an explicit expected model and a defined set of service and infrastructure relationships.

B. Static Architecture Conformance

Software Reflexion Models compare a high-level model with a source-derived model and distinguish convergence, divergence, and absence [9]. Knodel and Popescu compare several static conformance formulations [10], while Dependency Constraint Language expresses allowed and forbidden dependencies in object-oriented systems [11]. Model comparison and dependency rules are therefore established concepts, not our novelty claims.

Recent tools adapt conformance checking to particular languages and deployment settings. CALint checks Python implementations against the Clean Architecture dependency rule and accompanies detection with refactoring support [12]. ArchLintor is a particularly close language-level comparison: it detects architectural violations in TypeScript systems, reports module relationships, and supports CI/CD integration [13]. ArchSync observes typed service topology, not module interaction; no head-to-head comparison is reported.

IaC coupling-conformance metrics extend architecture checking to deployment practices [14], providing a relevant reference for ArchSync’s planned, unevaluated IaC extension. In a correction-oriented approach, 3Erefactor searches for executable refactorings that reduce architectural inconsistency [15]; ArchSync currently reports findings and gate outcomes rather than generating repairs. Language Models as Architectural Gatekeepers explores converting natural-language design rules into executable conformance tests [16]. It distinguishes executable tests from correctly expressed rules. ArchSync evaluates explicit rules deterministically; this does not imply greater accuracy.

Within the evaluated TypeScript service-topology slice, our contribution integrates established ideas into an evidence-linked, typed-relation, change-time contract; it is not an exhaustive novelty claim.

C. Change-Time Conformance

Architectural change can be examined at several time scales. Commit-level prediction relates source changes to architectural

changes [17]; a predicted change is not necessarily a violation of intended architecture. Repository mining combined with static reconstruction tracks microservice architectural evolution across development history [18]. One industrial case compares reconstructed view specifications with architecture models [8], while a conformance-visualization study reports practitioner feedback and challenges involving scalability, usability, and workflow integration [19]. Their different questions preclude interchangeable baseline scores.

ArchSync reports introduced findings through pull-request annotations and exit codes. Its co-developed benchmark establishes neither industrial adoption nor developer productivity.

D. Evaluation and Static-Analysis Reliability

Konersmann et al. examine evaluation and replication support in software-architecture research [20]. Abgaz et al.’s review concerns monolith-to-microservice decomposition [21]; that adjacent scope should not be treated as a direct measurement of architecture-conformance accuracy or proof that suitable conformance baselines do not exist. A more direct precedent is Schneider et al.’s comparison of static microservice-recovery tools: it identifies 13 tools, executes nine, and reports both individual and combined-tool recovery performance [22]. This is a baseline-study precedent, not a numerically comparable result without capability and ground-truth alignment.

Static-analysis-based visual models expose microservice dependencies at a system level [23]. Recovery from multiple dependency models explores how complementary relations affect reconstructed structure [24]. These motivate unevaluated multi-source extensions. Secondary syntheses are not independent trials of their underlying tools.

False positives and false negatives remain central to reliability. Interrogation testing distinguishes precision and soundness issues and demonstrates failures in mature analyzers [25]. ArchSync consequently includes annotated positive and hard-negative signals alongside topology changes. Manifests, pinned artifacts and replay support reproducibility, not representativeness. This synthesis establishes comparison dimensions, not exhaustive coverage or absence of prior work.

III. PROBLEM DEFINITION

Let the expected architecture contract be

$$M = (C_E, E_E, R), \quad (1)$$

where C_E is the set of expected components, E_E is the set of expected typed and directed relationships, and R is the set of conformance rules. We denote the graph projection of this contract by $G_E = (C_E, E_E)$. A relationship is represented as a tuple (u, t, v) , where u is the source component, v is the target component, and t is the relationship type.

Given a source-code repository S and supplied model M , a stack-specific analyzer A constructs an observed graph

$$G_O = A(S; M) = (C_O, E_O, P), \quad (2)$$

where C_O and E_O are observed components and relationships, and P maps each observation to source evidence. The analyzer uses expected component identifiers to map source paths and reuses declared metadata for matching identifiers. This is model-conditioned reconstruction, not autonomous discovery of architectural boundaries. Evidence contains file, line, column, detector, excerpt, and a fixed confidence annotation, not a calibrated correctness probability.

ArchSync supports four deterministic rule semantics. A *deny* rule rejects every matching relationship. An *allow* rule constrains the outgoing relationships of selected sources to an allowed target set. A *require* rule states that every matching source must have a direct edge to a matching target. A *require-path* rule states that every matching source must reach a matching target through a non-empty directed path. Rules may optionally constrain the relationship type and may use wildcard component selectors.

The conformance function compares M and G_O and returns a graph difference Δ , a set of rule violations $\mathcal{V}$, and a set of evidence-backed findings $\mathcal{F}$. Its classification is defined as

$$\text{class}(M, G_O) = \begin{cases} \text{violation,} & \text{if } |\mathcal{V}| > 0, \\ \text{evolution,} & \text{if } |\mathcal{V}| = 0 \wedge |\Delta| > 0, \\ \text{no-impact,} & \text{otherwise.} \end{cases} \quad (3)$$

For a Git change from base repository S_b to head repository S_h , ArchSync computes base and head finding sets and compares them by stable semantic identity:

$$\begin{aligned} \mathcal{F}_b &= \mathcal{F}(M, A(S_b; M)), \\ \mathcal{F}_h &= \mathcal{F}(M, A(S_h; M)), \\ K_b &= \{k(f) : f \in \mathcal{F}_b\}, \\ \mathcal{F}^+ &= \{f \in \mathcal{F}_h : k(f) \notin K_b\}. \end{aligned} \quad (4)$$

At the measured revision, $k(f)$ comprises kind, rule identifier, change, component, and canonical edge key; absent fields use empty strings. Source coordinates, messages, severity, and generated identifiers are not key fields. Thus an additional call site for an old violating edge need not introduce a finding. This is a relation-level policy, not an occurrence-level rejection policy.

Only $\mathcal{F}^+$ controls the pull-request decision. A new rule finding yields BLOCK; otherwise new evolution findings yield REVIEW; otherwise the result is PASS. Head findings with base keys are pre-existing; disappeared base keys are resolved. PASS certifies neither a violation-free head nor complete observation of source interactions.

The feasibility question is whether this model-conditioned analyzer and relation-level decision policy agree with declared controlled oracles. That agreement is a prerequisite for, not evidence of, effectiveness on independently labeled repository histories.

IV. RESEARCH QUESTIONS

RQ1: To what extent do model-conditioned graphs and detector signals agree with the declared D1/D2 annotations?

For the 20-patch development benchmark (D1), this question reports matched, extra, and missed graph elements and patch deltas. For the separate 40-signal corpus (D2), it reports descriptive precision, recall, F1, true negatives, and specificity against the annotated detector signals; these are controlled-corpus results, not population estimates.

RQ2: To what extent do classifications and violated-rule sets agree with declared controlled-case outcomes?

This question evaluates exact classification agreement with a manually curated ground truth and, for violations, agreement on the expected rule identifiers.

RQ3: To what extent do call-site and missing-relation anchor locations agree with declared file/line references at the case level?

This question separates localization of observed calls from deterministic anchors for missing relationships. It measures case-level file/line agreement, not recall of all findings or call sites. Column, detector, excerpt, and model path remain diagnostic evidence.

RQ4: Are ArchSync full-repository and Git-diff outputs deterministic and reproducible, and what analysis scope and latency are observed for the cached incremental gate?

This question compares normalized observed graphs, findings, and pull-request decisions across repeated executions and clean installations. For Git-diff checks it additionally records cache behavior, files parsed relative to the head repository, and cold/warm latency on the measured environment.

These scope-specific RQ formulations clarify reporting after the controlled runs; original inputs, labels, metrics, and acceptance criteria are unchanged.

V. PROPOSED APPROACH

A. Architecture as an Executable Contract

ArchSync uses a YAML document as the source of truth for architecture intent. The model defines stable component identifiers, component types, layers, directed relationships, rule severity, ownership metadata, and optional quality goals. JSON Schema validation rejects malformed documents, while semantic validation rejects unknown component references, duplicate edges, invalid identifiers, and other topology inconsistencies. Mermaid and draw.io views are derived from the model; they are not independent sources of truth.

Stable identifiers and canonical edge keys normalize the validated graph for shared command-line, benchmark and CI conformance semantics.

B. Source-to-Graph Analysis

A Guardian analyzer traverses TypeScript files under component roots and parses them through the TypeScript Compiler API. Rather than matching arbitrary text, detectors inspect syntax-tree nodes, imports, call expressions, variable initialization, environment-variable references, and URL literals. Stack-specific detectors translate these signals into typed architectural relationships.

The detector set recognizes HTTP requests, PostgreSQL clients and queries, Redis clients and operations, and AMQP

publishing and consumption. Version 0.2 tracks local bindings introduced by named, aliased, and namespace imports from `pg`, `redis`, and `amqplib`. PostgreSQL and Redis edges require an operation on an identifier associated with a recognized client initializer; AMQP edges require a channel associated with a tracked connection. These associations use per-file identifier names rather than complete compiler symbol or alias resolution. This package provenance filters same-named methods on unrelated objects when their receivers lack a tracked association; it does not establish soundness under arbitrary shadowing or reassignment. Endpoint resolution uses explicit literals and deterministic environment-variable fallbacks when available. Every detected relationship carries one or more source locations. Detected components and relationships are sorted and deduplicated before serialization.

C. Conformance and Decision Semantics

Source paths map to the longest expected component identifier that prefixes the relative path, falling back to the first path segment. URL targets use the lowercase hostname, ignoring port and path. Unresolved HTTP endpoints produce no edge and no explicit unknown finding for that call. Consequently, a deny-only contract can pass despite an unobserved interaction. AMQP publishing produces a producer-to-broker edge and consumption a broker-to-consumer edge. A typed require-path rule requires every traversed edge to have that type; an untyped rule permits mixed types. Broker-level reachability does not establish matching queues, routing keys, or message delivery.

The conformance engine first computes node and edge differences between the expected and observed graphs. It then evaluates the observed graph against the architecture rules. A rule violation takes precedence over ordinary topology evolution. This is a fixed conservative policy: one rule violation determines the top-level classification even when the change also contains acceptable evolution. Both types of findings remain available for inspection. Rule severity is reported but does not alter this precedence or the blocking threshold. This may delay acceptable changes bundled with a violation; alternative policies are not evaluated.

Full-repository checks map no-impact, violation and evolution to PASS, BLOCK and REVIEW, respectively; Git-diff checks apply this precedence only to introduced findings. The REVIEW outcome separates a request for a design decision from a detected rule breach. In the evaluated command, PASS, BLOCK, and REVIEW return exit codes 0, 1, and 3, respectively. A repository must configure its merge controls to treat the nonzero REVIEW result as requiring resolution; an annotation alone does not enforce approval. ArchSync reports the change without automatically modifying the approved architecture contract.

D. Diff-Scoped Living Architecture Loop

The Phase 3 gate resolves the merge base, enumerates modified, added, deleted, renamed, and untracked files, and maps changed TypeScript paths to source components. A cold check reconstructs the base Observed Graph from the committed

source tree. The graph is cached inside Git metadata under a key derived from the base commit, architecture-contract hash, and analyzer version. A warm check validates and reuses that cache.

For the proposed head, Guardian parses all TypeScript files in affected source components, rather than only changed lines, retaining independently detectable relationships elsewhere in those components. This scope does not provide inter-file client/data-flow resolution. Those components' observations replace their baseline counterparts; unaffected components remain cached. The same supplied model is used for analysis and conformance at base and head; separate historical models are not loaded. Editing the model invalidates the cache and changes the contract used for both snapshots, not a model-to-model finding delta. Cache-envelope checks assume trusted local contents and a maintained analyzer version; they do not authenticate the graph. Annotations, a report, and exit codes expose the decision.

The analyzer updates observations, never the model. Owner-reviewed model changes are a proposed acceptance workflow, not authenticated approvals in the evaluated gate. It does not persist accepted/rejected review decisions. If evolution is rejected but code and contract stay unchanged, the next run returns REVIEW again, not BLOCK. Repository controls must prevent merging; the team revises code or contract and reruns. Rationale recording and new rules for lasting constraints remain manual governance.

E. Evidence-Rich Findings

A finding contains a contract version, identifier, kind, severity, message, optional rule, affected element, source evidence, and model evidence. A forbidden edge retains recognized supporting call sites, not necessarily a newly added occurrence. A missing required edge or path uses a deterministic component anchor and the model rule: an absent call has no call site to localize.

End-to-end example. Existing D1 case 06 adds a frontend `fetch` with a payment-service URL fallback at `frontend/src/app.ts:14`. Source and hostname mapping yield `(frontend,http,payment-service)`. The edge matches deny rule ARCH-001. Its key is absent at the base, so the linked rule and call evidence yield BLOCK even though an evolution finding is also present. This is an existing controlled case, not an additional evaluation sample.

VI. SYSTEM ARCHITECTURE

Figure 1 separates approved intent from base and proposed implementation state. ArchSync Core validates the model and provides the expected graph and executable rules. Guardian caches a base Observed Graph, scopes the Git diff to affected source components, incrementally constructs the head graph, and preserves source evidence. The gate compares base and head findings before deriving the pull-request decision.

The architecture is divided across focused repositories. `archsync-core` contains the architecture contract and deterministic conformance functions. `archsync-guardian` contains

analyzers and findings. `archsync-benchmark` contains the controlled development benchmark. `archsync-examples` contains readable models and generated views. `archsync-mcp` is reserved for a later interaction layer and is excluded from the initial experiment. The named draft exposes these real repository links; the double-blind build replaces them with blinded artifact labels.

VII. IMPLEMENTATION

A. Prototype and Detector Logic

`archsync-core` implements the architecture contract, schema and semantic validation, normalized graph comparison, and deny/allow/require/require-path rules. `archsync-guardian` implements TypeScript extraction, source evidence, and the Git-diff gate. Guardian delegates conformance semantics to Core so repository and incremental checks use the same rule engine.

For a concrete PostgreSQL example, consider the following code inside the `order-service` component:

```
import { Pool as PgPool } from "pg";
const db = new PgPool({
  connectionString:
    "postgres://postgres:5432/orders"
});
await db.query("select 1");
```

The import detector records `PgPool` as a local name for a recognized `pg` constructor. The initializer pass recognizes `new PgPool`, resolves the inline `connectionString`, and associates `db` with target `postgres`, derived from the URL hostname. At `db.query`, the call detector requires that receiver association before emitting the directed edge (`order-service,data,postgres`). Merely importing `pg`, constructing an unrelated client, or calling an unrelated object's `query` method does not satisfy this detector. Construction alone does not emit the database-access edge. Evidence identifies the query call's file, line, column, detector, and excerpt.

The analysis uses per-file identifier maps and four passes over variable declarations; it is not general alias analysis or interprocedural data-flow analysis. Imported client wrappers, arbitrary reassignment, shadowed identifiers, and relationships requiring resolution across files are not comprehensively handled. Endpoint resolution uses literals, mapped variables, and naming conventions such as `DATABASE_URL` for `postgres`. For binary expressions it tries the right-hand expression first; this deterministic heuristic does not evaluate runtime environment values or branch conditions. The PostgreSQL evidence confidence of 0.99 is a fixed detector annotation, not a calibrated probability. Redis operations similarly require a recognized client, and AMQP operations require a channel associated with a recognized connection.

B. Controlled Evaluation Artifact

`archsync-benchmark` supplies the synthetic Order Platform, 20 executable patches with group-authored ground truth, and the 40-signal positive/hard-negative corpus. Each patch runs

TABLE I
RELATION-LEVEL GATE BOUNDARIES. OUTCOMES DEPEND ON SUPPORTED OBSERVATIONS; THEY DO NOT GUARANTEE COMPLETE DETECTION.

Condition	Gate behavior	What remains outside the guarantee
New observed rule-finding key	BLOCK	Unsupported or unresolved interactions.
New evolution, no new rule finding	REVIEW	Human approval or automatic rejection handling.
No new key on a drifting base	PASS; old findings remain visible	A violation-free head.
New call site for an old violating edge	Can PASS; key unchanged	Rejection of every violating occurrence.
Unresolved HTTP target	No edge or explicit unknown finding	Complete observation of communication.
Code and model edited together	Same model evaluates both snapshots	Authorization of the changed contract.

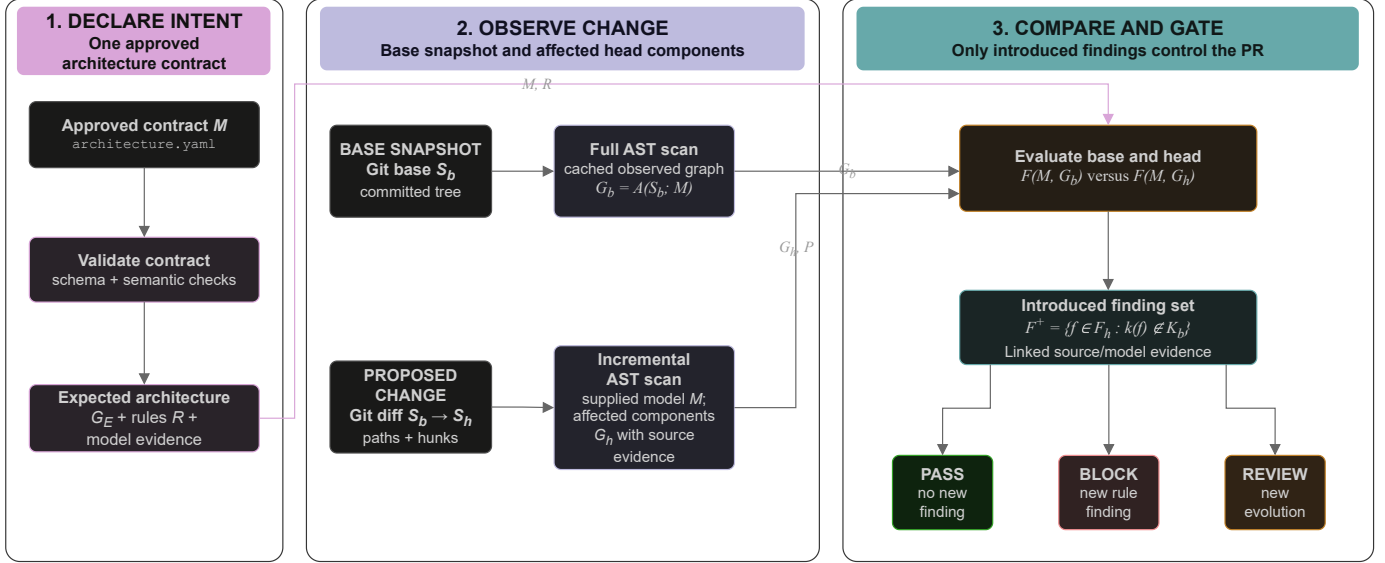


Fig. 1. ArchSync Phase 3 system architecture. The three-stage pipeline separates approved intent, base/head observation, and the pull-request gate. The model remains authoritative; only findings introduced by the Git diff determine PASS, BLOCK, or REVIEW.

on a fresh copy. Git-diff experiments additionally construct a baseline commit, perform cold and warm checks, and compare incremental head observations with a separate full scan. This full scan checks incremental equivalence; it does not independently establish extraction accuracy.

Measurements use Core commit 2affbbb0da85, Guardian commit 8779bf7965b3, and Benchmark commit 24d63ebf2fc3. Later development heads do not replace these measured artifacts. Source/patch manifests, raw results, package hashes, historical verification commands, and the detector/gate audit remain in the supplementary artifact. These controls support inspection, not independent accuracy evidence. archsync-mcp is excluded.

VIII. CONTROLLED VERIFICATION METHODOLOGY

A. Experimental Objects

The evaluation uses two controlled TypeScript/Node.js datasets. Dataset D1 is an end-to-end Order Platform benchmark whose expected baseline contains five components and five directed relationships. Twenty patches are distributed as nine no-impact changes, seven rule violations, and four architecture evolutions. In addition to refactoring, service

bypass, required-edge removal, Redis, and AMQP cases from the initial benchmark, the expansion includes documentation-only URLs, type-only imports, method-name lookalikes, direct gateway database access, allow-rule violations, a broken multi-hop path, aliased PostgreSQL construction, a new service, and a Redis hash operation.

Each D1 case declares changed files, expected graph delta, classification, findings, and source location. The topology labels and classifications originated in Phase 1. Patch implementations and exact evidence lines were refined while the Phase 2 analyzer was integrated. D1 is therefore a development benchmark and regression oracle, not a blinded holdout. Validators check structural consistency, distribution, patch applicability, changed-file membership, patch-hunk containment, and integrity hashes; they do not replace independent semantic review.

Dataset D2 is a detector challenge corpus with five groups: fetch, PostgreSQL, Redis, AMQP publish, and AMQP consume. Each group contains four positive and four hard-negative source lines, giving 20 positives and 20 negatives. Positive variants include aliases, namespace imports, environment fallbacks, and multiple supported operations. Hard

negatives include URL text without a call, custom functions, unused package imports, local objects with detector-shaped method names, lifecycle calls, comments, and string literals. The manifest binds each signal to a detector, source file, exact line, expected edge, description, and required source substring.

D2 was authored after the initial v0.1 analyzer and deliberately targets its weaknesses. We execute both the frozen v0.1 artifact and provenance-aware v0.2 on the same corpus. This paired comparison identifies concrete regression behavior but is not an unbiased estimate of improvement on unseen programs.

The Phase 3 protocol reuses D1 rather than introducing a third dataset. For each case, it creates a fresh Git repository from the unchanged baseline, commits that baseline with fixed author metadata, applies one patch to the working tree, and runs the diff gate twice. The first execution reconstructs and caches the base graph; the second must hit the cache. A separate full scan of the same patched head acts as an equivalence oracle for the incremental result. Ground truth is extended operationally with expected changed files and the PASS/BLOCK/REVIEW mapping already implied by each D1 label. We compare normalized cold and warm results, decisions, changed-file sets, exact architecture deltas, rule identifiers, evidence locations, and incremental/full-scan equivalence. Timings are retained only from the recorded Windows run because shared CI runners do not provide a controlled performance environment.

Figure 2 summarizes the executable verification protocol and shows how raw analyzer executions become the reported metrics and evidence artifact.

B. External-Comparison Boundary

External comparison remains future work; it was not executed within the reported evaluation. No accepted comparator result or common label mapping is available. Module imports and remote service calls are different observation units. A module-dependency analyzer is therefore only a candidate, not an established equivalent baseline.

Before execution, the proposed protocol requires frozen repositories and revisions, tool versions and configurations, dependency-resolution conditions, independently reviewed labels, and a common scoring unit. It rejects a comparator unless a non-empty semantic intersection is established; any adapter must be separately frozen before execution. Unsupported relations are reported separately, not counted as false negatives. Reusing D1 would remain a controlled comparison, not independent validation. These requirements contribute no observations to the results.

C. Procedure

D1 validates model, ground truth, hashes and patch applicability before execution. The unchanged baseline is analyzed twice. Each of the 20 patches is then applied alone in a fresh temporary baseline copy, analyzed twice and compared with its declared graph delta, classification, rule identifiers and

locations. Copies are removed between cases. This yields 42 analyzer executions.

D2 validates its annotated source lines and analyzes the corpus twice with each pinned version: four further executions, or 46 across the current and historical Phase 2 evidence. The continuously rerun v0.2 gate executes 44; the frozen v0.1 package and result are verified without rewriting that historical observation. Observations are expanded to detector–edge–file–line signals for scoring.

P3 executes 40 checks (cold and warm for each D1 case). A cold check analyzes base and affected components; a warm check reuses the base cache. Twenty separate full-head scans provide the equivalence oracle: 80 analyzer calls in total. Normalized comparison excludes elapsed time and cache-hit state but retains decisions, classifications, changed files, affected components, findings, graph delta, evidence and scan scope.

Repeated results are compared as normalized JSON. The benchmark workflow installs SHA-256-verified runtime packages and executes the Phase 2 and Phase 3 protocols plus the PASS/BLOCK/REVIEW quickstart from clean `ubuntu-latest` and `windows-latest` checkouts; both jobs passed for benchmark audit commit `5fe3679`.

The original timing artifact records Node.js 22.16.0, Windows OS release `10.0.26200`, an Intel Core i7-13650HX, and 20 logical CPUs. Cases ran sequentially. RAM, power mode, background load, thermal state, and filesystem-cache state were not recorded; present-day settings cannot fill those historical gaps.

D. Metrics

D1 reports counts of matched, extra, and missed graph items, with changed items separated from repeated baseline structure. For D2 only, TP and FN count annotated positive detector signals, and TN and FP count explicitly annotated hard negatives. Its descriptive precision, recall, F1, and specificity are

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (5)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (6)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (7)$$

$$\text{Specificity} = \frac{TN}{TN + FP}. \quad (8)$$

We report both complete-graph and patch-changed node and edge counts. When both expected and predicted changed sets are empty, the case is an exact match but contributes no positive item to the aggregate TP/FP/FN counts. D2’s TP, FP, FN, and TN units are annotated detector signals rather than distinct graph edges; several positive source locations may support the same edge.

Classification agreement uses all 20 cases; rule-set agreement uses seven violations. The 11 finding-bearing cases are decomposed post hoc into nine call-site cases and two missing-relation anchor cases. A match requires the declared file and

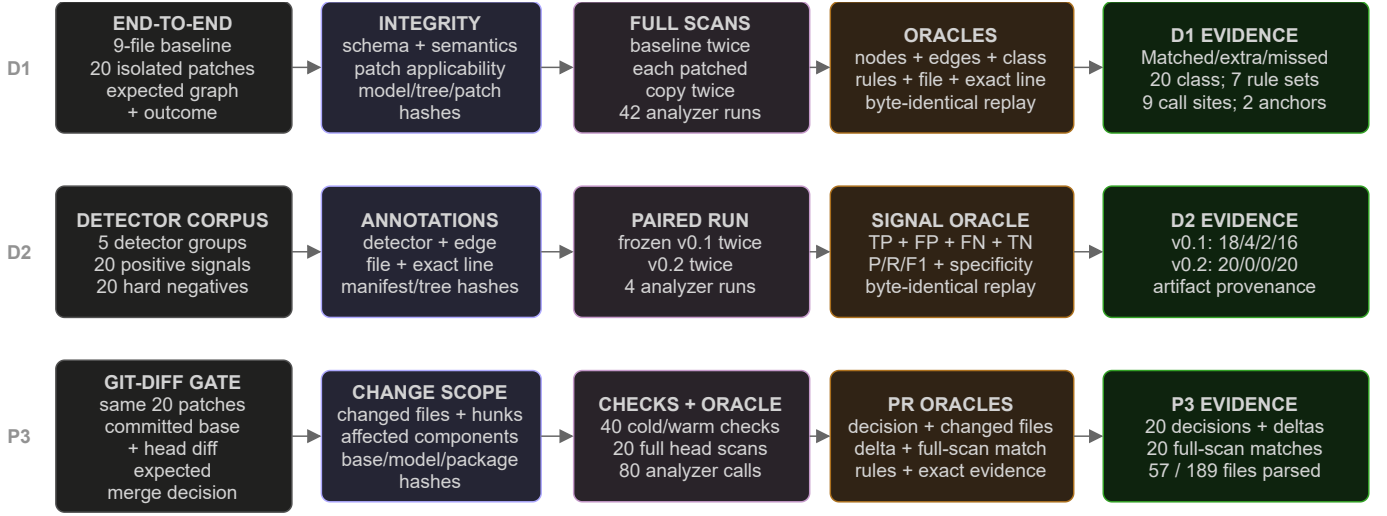


Fig. 2. Controlled verification protocol. D1 measures full-repository reconstruction and classification, D2 isolates detector signals, and P3 replays D1 as cold and warm Git-diff pull-request checks. All three routes use co-developed development data.

line at the case level, not recall over all findings or call sites. P3 locations are matched by finding kind and rule identifier or evolution endpoints; the historical P2 artifact retains aggregated matched-finding evidence. The decomposition changes reporting, not labels or the original acceptance gate. Its derivation and input hashes are retained in the supplementary artifact. Determinism requires byte-identical normalized output across duplicate executions.

P3 additionally measures exact merge-decision, changed-file-set, architecture-delta, and incremental/full-scan agreement. Incremental scope is

$$r_{\text{parse}} = \frac{\sum_i |T_i^{\text{parsed}}|}{\sum_i |T_i^{\text{head}}|}, \quad (9)$$

where T_i^{parsed} contains TypeScript files parsed in affected components and T_i^{head} contains TypeScript files represented by the reconstructed head graph. Cache correctness requires a miss followed by a hit and identical normalized results. Latency uses the recorded nearest-rank percentile $x_{(\lceil pn \rceil)}$ at $p = 0.5$ and 0.95 . For $n = 20$, $p50$ is the tenth observation, not the average of observations ten and eleven. Original raw values are preserved; no inferential comparison is made.

E. Engineering Acceptance Criteria

The D1 prototype gate required complete/changed graph precision and recall of at least 0.85, exact case classifications and violated-rule sets, matching evidence locations, deterministic replay and repository coverage thresholds. D2 v0.2 required all annotated positives detected, no unexpected detections, all hard negatives rejected and byte-identical replay. P3 required exact applicable decisions, deltas, changed files, rules and locations, plus full-scan equivalence, cache/replay agreement and cross-platform functional completion. The 0.85 threshold is an engineering target, not a statistical standard; passing verifies the declared regression contract only.

TABLE II
D1 COUNTS ON 20 CO-DEVELOPED PATCHES. THE FIVE CHANGED NODES AND 12 CHANGED EDGES ARE REGRESSION ITEMS, NOT INDEPENDENT SAMPLES OF GENERAL ACCURACY; FULL-GRAPH COUNTS REPEAT BASELINE STRUCTURE.

Measure	Matched	Extra	Missed
Full-graph nodes	105	0	0
Full-graph edges	108	0	0
Changed nodes	5	0	0
Changed edges	12	0	0

TABLE III
DECISION, EVIDENCE, AND REPLAY OUTCOMES ON CONTROLLED D1 CASES.

Measure	Matches	Total
Classification	20	20
Violation rule set	7	7
Call-site file and line	9	9
Missing-relation anchor file and line	2	2
Deterministic cases	20	20

IX. CONTROLLED VERIFICATION RESULTS

RQ1: graph reconstruction and detector signals. The unchanged repository was reconstructed as five components and five relationships with a no-impact classification. Table II reports no false positive or false negative in D1. Full-graph totals aggregate 105 expected nodes and 108 expected edges across 20 patched repositories. Only five node changes and 12 edge changes contribute positive changed-graph items. These finite, co-developed cases test contract agreement; they do not estimate performance on unseen projects.

Table V gives the signal-level regression result. On D2, v0.1 achieved precision 0.818, recall 0.900, F1 0.857, and specificity 0.800. Its four false positives were a PostgreSQL-shaped local query, two AMQP-publish lookalikes, and an AMQP-consume lookalike; its two false negatives were Redis hSet

TABLE IV
P3 GIT-DIFF GATE OUTCOMES AND MEASURED SCOPE ON THE D1 REPLAY.

Measure	Result
Classification agreement	20/20
Merge-decision agreement	20/20
Changed-file agreement	20/20
Architecture-delta agreement	20/20
Incremental/full-scan agreement	20/20
Violation rule-set agreement	7/7
Call-site exact line	9/9
Anchor exact line	2/2
Cache hit on repeated check	20/20
Normalized cold/warm replay	20/20
Incremental files / head files	57/189
Cold p50 / p95 (ms)	518.51 / 531.05
Warm p50 / p95 (ms)	242.62 / 249.30

and `lPush`. On the same annotated signals, v0.2 matched all 20 positives and rejected all 20 annotated negatives (precision, recall, F1, and specificity each 1.000 on this corpus). Package-provenance tracking removed the four method-name false positives, while the expanded Redis operation set recovered the two missed positives. Because D2 was constructed from known v0.1 weaknesses, this is targeted regression evidence and not an estimate over arbitrary TypeScript syntax or an external baseline comparison.

RQ2: change classification. ArchSync matched all 20 D1 labels: nine no-impact, seven violation, and four evolution cases. It also matched the complete expected rule-identifier set for all seven violations. The added allow and require-path cases exercise rule semantics beyond direct deny and require checks. Redis, AMQP, Fraud Service, and payment-cache additions were classified as evolution rather than violation, preserving a human-review path.

RQ3: evidence localization. All 11 finding-bearing D1 cases are split post hoc into call-site file/line agreement in 9/9 cases and anchor agreement in 2/2 cases, for missing-edge case 08 and missing-path case 17. Both P2 and P3 satisfy the separate checks. These are case-level outcomes: case 15 has two violated rules but contributes one case. An anchor is not a localized absent call, and neither measure evaluates ranking, all supporting locations, or reviewer effort.

RQ4: determinism and reproducibility. The D1 baseline and all 20 cases produced byte-identical normalized JSON across duplicate executions, giving 42 current-version executions with no replay mismatch. Both D2 analyzer versions were also deterministic across their duplicate runs. P3 produced identical normalized cold and warm results for 20/20 cases, matched all 20 declared architecture deltas, matched a separate full head scan in all 20 cases, and hit the baseline cache on every repeated check. As Table IV shows, component-incremental analysis parsed 57 of 189 TypeScript file instances, $r_{\text{parse}} = 0.3016$. On the recorded Windows environment, warm nearest-rank p50 latency was 242.62 ms compared with 518.51 ms cold, a descriptive 53.2% reduction; warm p95 was 249.30 ms compared with 531.05 ms cold. Local verification reproduced all stored results and repository

gates. GitHub Actions independently reproduced the functional benchmark, detector, and Git-diff evidence on both `ubuntu-latest` and `windows-latest`; timing values were not treated as cross-platform acceptance thresholds.

Boundary of the reported evidence. The observations above come from D1, D2, and the P3 reuse of D1. No external-tool result is accepted into this evaluation, and no common label mapping has been established for comparative scoring. The internal v0.1–v0.2 detector comparison and incremental/full-scan equivalence therefore answer different questions from an external baseline: the former exposes targeted regression behavior, while the latter checks whether scoped execution preserves this implementation’s full-scan result.

Exact agreement with these oracles does not establish that the expected architecture is complete or appropriate for another system. In particular, a missed relationship outside the detector vocabulary can be absent from both incremental and full scans without producing an equivalence mismatch. Similarly, duplicate executions can reproduce the same incorrect inference. The graph, evidence-location, decision, and replay checks are complementary consistency checks; they do not substitute for independently authored semantic labels or a separately evaluated tool.

X. DISCUSSION

The controlled verification links declared architecture, supported observations and change-time decisions. D1 checks isolated graph and rule changes; D2 exposes detector look-alikes and unsupported operations; P3 checks incremental/full-scan agreement. Shared implementation and development data prevent treating these complementary checks as independent accuracy confirmations.

The relevant denominators are five changed nodes, 12 changed edges, seven violation rule sets, nine call-site cases and two anchor cases, and 20 positive plus 20 hard-negative detector signals. Repeated baseline structure inflates full-graph totals without increasing the diversity of changed relationships. D2 was deliberately designed from v0.1 weaknesses and informed v0.2. Its value is targeted diagnosis and regression protection. It is not a competitor baseline. A stronger comparison needs independent labels and a reviewed common observation unit; deterministic replay can reproduce the same wrong inference.

The relation-level gate permits gradual adoption: pre-existing keys remain visible without blocking unrelated changes. That choice also means an additional violating occurrence on the same edge may pass. The policy is therefore appropriate only when teams intend to gate new relationship-level findings, not every new violating call. An unresolved endpoint can escape observation altogether. Table I makes these limits explicit so PASS cannot be mistaken for a complete architectural safety guarantee.

Separating violation from evolution avoids automatically forbidding every new infrastructure component. It does not automate architecture authority. The measured command neither authenticates a model-change approval nor records a durable

TABLE V
D2 WITHIN-TOOL REGRESSION CONFUSION MATRICES FOR THE FROZEN v0.1 AND PROVENANCE-AWARE v0.2 ANALYZERS ON THE SAME CO-DEVELOPED 40-SIGNAL CORPUS.

Detector	v0.1				v0.2			
	TP	FP	FN	TN	TP	FP	FN	TN
fetch	4	0	0	4	4	0	0	4
PostgreSQL	4	1	0	3	4	0	0	4
Redis	2	0	2	4	4	0	0	4
AMQP publish	4	2	0	2	4	0	0	4
AMQP consume	4	1	0	3	4	0	0	4
Overall	18	4	2	16	20	0	0	20

rejection. A human rejection leaves unchanged inputs at REVIEW; repository controls must prevent the merge while code or contract changes are resolved. Later governance modules and protected branches cannot retrospectively establish the effectiveness of the measured workflow.

The observed 57/189 parsed-file ratio describes input scope, not CPU savings. The 53.2% cold-to-warm nearest-rank p50 reduction combines parsing, Git startup, cache I/O, comparison, and reporting. Sequential observations on one small system do not isolate those costs or establish scalability.

External comparison requires frozen independent repositories and a capability-matched comparator. Labels must cover files without predictions to measure recall. Report per-repository errors, unsupported-vocabulary coverage and failures. The supplementary technical-validity extension specifies metamorphic and implementation mutation checks; these remain development evidence. Practitioner evaluation must separately measure false-block burden and review effort. Regression runs and citations cannot substitute for independent validation.

XI. THREATS TO VALIDITY

Construct validity. Typed edges and explicit rules measure a selected structural view, not component responsibilities, rationale, runtime quality attributes, or developer usefulness. Reconstruction is model-conditioned: expected component identifiers and metadata help define the observed view. An omitted relationship can disappear from both incremental and full scans without an equivalence failure. File/line agreement is case-level and split between observed call sites and deterministic missing-relation anchors; it measures neither completeness/ranking of evidence nor reviewer comprehension. D2 specificity covers its declared hard negatives, not all unsupported syntax. Unsupported-vocabulary coverage needs a separate denominator. Parsed-file counts are not CPU-work measurements.

Internal validity. The authors developed both the analyzer and datasets. D1 topology intent preceded integration, but executable patches and exact locations co-evolved with the detector. P3 reuses those known patches. D2 was authored from v0.1 errors and guided v0.2, so tuning and confirmation risks remain. Isolated patches, raw predictions, integrity manifests and duplicate execution expose inconsistency but do not establish independent semantic labels. The call-site/anchor

split and scope-specific RQ wording are post-hoc reporting clarifications; frozen labels and acceptance gates were not changed.

External validity. One synthetic TypeScript/Node.js system with 20 patches and 40 detector signals cannot represent production repositories or longitudinal histories. Wrappers, dynamic endpoints, inter-file flows, shadowing, dependency injection, generated code, unconventional monorepos, other languages, IaC, and runtime-only relations remain unevaluated or incompletely observed. Hostname-level targets ignore port and path; AMQP broker paths do not prove routing compatibility. No industrial deployment or developer study was performed. Repository protection introduced later is not evidence of governance effectiveness in this evaluation.

Comparative validity. No external comparison was executed within the reported evaluation, and no accepted comparator result or common label mapping is available. v0.1 versus v0.2 is within-tool regression only. Module imports and remote service calls are different units unless an independently reviewed mapping establishes a shared abstraction. Unsupported relationships must not become comparator false negatives. Tool versions, configurations, dependency resolution, common labels and scoring units must be frozen before interpreting comparative results.

Literature-positioning validity. Related Work is a purposively scoped narrative synthesis, with source-selection and interpretation bias. It may omit relevant work; overlapping projects are not independent replications. These are limitations of the chosen narrative method, not temporary gaps awaiting completion of another review. Neither exhaustive coverage nor absence of competitors is established. The supplementary access register distinguishes metadata, available full texts, extracted evidence and pending human appraisal. Citation recency and DOI identity alone do not establish full-text verification or venue quality.

Conclusion validity. The few changed graph items and targeted signals are finite regression objects, not independent random samples from a defined repository population. Accordingly, no population accuracy estimate, significance test or confidence interval is inferred from perfect agreement. Latency p50/p95 are descriptive nearest-rank statistics on one recorded environment, not a causal incremental-analysis effect or a scalability estimate.

Reliability. Inputs, runtime archives and normalized outputs

are pinned and hashed. Each current input is run twice; P3 checks cache miss/hit and full-scan agreement. The older detector result remains provenance-validated rather than continuously regenerated. Historical Windows and Linux functional replays support portability of the controlled protocol, while latency remains the original Windows recording. Unrecorded machine-load conditions and a trusted local graph-cache assumption limit reproducibility. Later software changes are separated from the revisions that produced these observations.

XII. CONCLUSION AND FUTURE WORK

ArchSync deterministically links declared contracts and source-derived TypeScript service graphs. Controlled outputs match the declared regression oracles, and provenance tracking corrects earlier detector errors on the targeted corpus.

They do not establish general accuracy, comparative advantage, or governance effectiveness. Data co-evolved with the implementation; accepted external comparison and common label mapping are absent, and no independent real-world hold-out exists. The analyzer never rewrites the model; evolution needs human review. Future gates are an independent multi-repository holdout, capability-matched external comparison, and real pull-request pilot. Only after those gates produce auditable evidence can external accuracy, performance, or developer value be claimed.

REFERENCES

- [1] R. Li, P. Liang, M. Soliman, and P. Avgeriou, "Understanding software architecture erosion: A systematic mapping study," *Journal of Software: Evolution and Process*, vol. 34, no. 3, p. e2423, 2022.
- [2] E. Anthony, A. Berntsson, T. Santilli, and R. Wohlrab, "We're drifting apart: Architectural drift from the developers' perspective," in *2024 IEEE 21st International Conference on Software Architecture*. Hyderabad, India: IEEE, 2024, pp. 101–111.
- [3] R. Li, M. Soliman, P. Liang, and P. Avgeriou, "Symptoms of Architecture Erosion in Code Reviews: A Study of Two OpenStack Projects," in *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. IEEE, 2022, pp. 24–35.
- [4] R. Li, P. Liang, and P. Avgeriou, "Warnings: Violation symptoms indicating architecture erosion," *Information and Software Technology*, vol. 164, p. 107319, 2023.
- [5] S. Ducasse and D. Pollet, "Software architecture reconstruction: A process-oriented taxonomy," *IEEE Transactions on Software Engineering*, vol. 35, no. 4, pp. 573–591, 2009.
- [6] A. Qayum, M. Zhang, S. Colreavy, M. Chochlov, J. Buckley, D. Lin, and A. R. Sai, "A Framework and Taxonomy for Characterizing the Applicability of Software Architecture Recovery Approaches: A Tertiary-Mapping Study," *Software: Practice and Experience*, 2024, first published online 17 July 2024.
- [7] A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, and D. Taibi, *Tools Reconstructing Microservice Architecture: A Systematic Mapping Study*. Springer Nature Switzerland, 2024, pp. 3–18. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-66326-0_1
- [8] B. Uzun and B. Tekinerdogan, "Detecting deviations in the code using architecture view-based drift analysis," *Computer Standards & Interfaces*, vol. 87, p. 103774, 2024.
- [9] G. C. Murphy, D. Notkin, and K. J. Sullivan, "Software reflexion models: Bridging the gap between source and high-level models," in *Proceedings of the Third ACM SIGSOFT Symposium on Foundations of Software Engineering*. New York, NY, USA: ACM, 1995, pp. 18–28.
- [10] J. Knodel and D. Popescu, "A comparison of static architecture compliance checking approaches," in *Sixth Working IEEE/IFIP Conference on Software Architecture*. Los Alamitos, CA, USA: IEEE, 2007, pp. 12–21.
- [11] R. Terra and M. T. Valente, "A dependency constraint language to manage object-oriented software architectures," *Software: Practice and Experience*, vol. 39, no. 12, pp. 1073–1094, 2009.
- [12] C. O. de Oliveira Junior, J. Carvalho, F. F. Silveira, T. S. da Silva, and E. M. Guerra, "CALint: A Tool for Enforcing the Clean Architecture's Dependency Rule in Python," in *Computational Science and Its Applications – ICCSA 2022 Workshops: Malaga, Spain, July 4–7, 2022, Proceedings, Part V*. Springer International Publishing, 2022, pp. 534–549.
- [13] M. Resende, R. Durelli, and R. Terra, "ArchLintor: detecting architectural violations in TypeScript systems," in *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*, ser. SBES 2025. Sociedade Brasileira de Computação, Sep. 2025, pp. 900–906. [Online]. Available: <http://dx.doi.org/10.5753/sbes.2025.11075>
- [14] E. Ntontos, U. Zdun, J. Soldani, and A. Brogi, *Assessing Architecture Conformance to Coupling-Related Infrastructure-as-Code Best Practices: Metrics and Case Studies*. Springer International Publishing, 2022, pp. 101–116. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-16697-6_7
- [15] J. Liu, W. Jin, J. Zhou, Q. Feng, M. Fan, H. Wang, and T. Liu, "3Erefactor: Effective, Efficient and Executable Refactoring Recommendation for Software Architectural Consistency," *IEEE Transactions on Software Engineering*, vol. 50, no. 10, pp. 2633–2655, Oct. 2024. [Online]. Available: <http://dx.doi.org/10.1109/tse.2024.3449564>
- [16] A. Lucena, E. L. G. Alves, and J. Brunet, "Language Models as Architectural Gatekeepers: Automating Conformance Checking from Natural Language," in *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*. Sociedade Brasileira de Computação, 2025, pp. 713–719.
- [17] W. Zhan, R. Mo, Y. Jiang, and D. Wang, "Just-in-Time Prediction of Software Architectural Changes Through Commit-Level Analyses," *IEEE Transactions on Software Engineering*, vol. 51, no. 8, pp. 2285–2304, Aug. 2025. [Online]. Available: <http://dx.doi.org/10.1109/tse.2025.3587849>
- [18] P. Genfer and U. Zdun, "Exploring Architectural Evolution in Microservice Systems Using Repository Mining Techniques and Static Code Analysis," in *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, September 3–6, 2024, Proceedings*. Springer Nature Switzerland, 2024, pp. 157–173.
- [19] F. Zamfirov, A. Radulescu, J. Krüger, and M. R. V. Chaudron, "Lessons from Visualizing Software Architecture Structure Conformance at Thermo Fisher Scientific," in *Software Engineering and Advanced Applications: 51st Euromicro Conference, SEAA 2025, Salerno, Italy, September 10–12, 2025, Proceedings, Part III*. Springer Nature Switzerland, 2025, pp. 372–389.
- [20] M. Konersmann, A. Kaplan, T. Kühn, R. Heinrich, A. Koziol, R. H. Reussner, J. Jürjens, M. al Doori, N. Boltz, M. Ehl, D. Fuchß, K. Großer, S. Hahner, J. Keim, M. Lohr, T. Sağlam, S. Schulz, and J.-P. Töberg, "Evaluation methods and replicability of software architecture research objects," in *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. Honolulu, HI, USA: IEEE, 2022, pp. 157–168.
- [21] Y. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz, M. Bivol, G. Jackson, M. Yilmaz, J. Buckley, and P. Clarke, "Decomposition of monolith applications into microservices architectures: A systematic review," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, 2023.
- [22] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, and D. Taibi, "Comparison of static analysis architecture recovery tools for microservice applications," *Empirical Software Engineering*, vol. 30, no. 5, Jun. 2025. [Online]. Available: <http://dx.doi.org/10.1007/s10664-025-10686-2>
- [23] T. Cerny, A. S. Abdelfattah, J. Yero, and D. Taibi, "From static code analysis to visual models of microservice architecture," *Cluster Computing*, vol. 27, no. 4, pp. 4145–4170, Apr. 2024. [Online]. Available: <http://dx.doi.org/10.1007/s10586-024-04394-7>
- [24] M. Altunışık, H. Sözer, and G. Gürsun, "Software Architecture Recovery from Multiple Dependency Models," in *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*. ACM, 2024, pp. 1185–1192.
- [25] D. Kaindlstorfer, A. Isychev, V. Wüstholtz, and M. Christakis, "Interrogation testing of program analyzers for soundness and precision issues," in *Proceedings of the 39th IEEE/ACM International Conference on*

Automated Software Engineering. New York, NY, USA: ACM, 2024,
pp. 319–330.